

Adder.sv

```
module Adder
(
input logic [2:0] SW, // 3 switches used as inputs: SW[0]=a, SW[1]=b,
SW[2]=cin
output logic [1:0] LEDR // LEDs: LEDR[0]=sum, LEDR[1]=carry out
);

// Instantiate FullAdder
// Connect switches to inputs and LEDs to outputs
FullAdder i0(SW[0], SW[1], SW[2], LEDR[0], LEDR[1]);

endmodule
```

HalfAdder.sv

```
module HalfAdder
(
input logic a, // First input bit
input logic b, // Second input bit
output logic sum, // Sum output (XOR of inputs)
output logic cout // Carry output (AND of inputs)
);

// always_comb means this block automatically reacts to any input change
// and describes purely combinational logic (no memory, no clock)
always_comb
begin
sum = a ^ b; // XOR gives sum without carry
cout = a & b; // AND gives carry when both inputs are 1
end

endmodule
```

FullAdder.sv

```
module FullAdder
(
input logic a, // First input bit
```

```

input logic b, // Second input bit
input logic cin, // Carry-in input
output logic sum, // Final sum output
output logic cout // Final carry output
);

// Internal signals
logic c0; // Carry from first half adder
logic c1; // Carry from second half adder
logic s0; // Intermediate sum from first half adder

// First half adder: adds a and b
HalfAdder i1(a, b, s0, c0);

// Second half adder: adds intermediate sum and carry-in
HalfAdder i2(s0, cin, sum, c1);

// Final carry is OR of both carry outputs
assign cout = c0 | c1;

endmodule

```

FullAdder_TB.sv

```

// Defines simulation time units and precision
`timescale 1ns / 1ps

module FullAdder_TB();

    // Testbench signals (inputs and outputs)
    logic a;
    logic cin;
    logic b;
    logic sum;
    logic cout;

    // Instantiate the Unit Under Test (UUT)
    // Named connections improve readability
    FullAdder uut(
        .a(a),
        .cin(cin),

```

```

        .b(b),
        .sum(sum),
        .cout(cout)
    );

    initial
    begin
        // Initialise all inputs to 0 at start of simulation
        a = 0;
        cin = 0;
        b = 0;

        // $monitor prints values whenever ANY listed signal changes
        $monitor("a=%0b b=%0b cin=%0b sum=%0b cout=%0b", a, b, cin, sum,
cout);

        // Loop through all 8 possible input combinations (3 bits → 2^3 = 8)
        for (int i = 0; i < 8; i++)
        begin
            // Assign bits of i to a, b, cin
            // {a, b, cin} means:
            // a = MSB, cin = LSB of i
            {a, b, cin} = i;

            #10; // Wait 10 ns before next input combination
        end
    end

endmodule

```